

Draft-Strategic Data Masking & Classification

Document Control

Document Status

Date	Version	tatus
24-Feb-2026	v.0	

Document Authors

Name	Role
Naresh Kala	Platform Team Member
Koustubh Kasalkar	Offshore Lead
Syed Fayaz	Onshore Lead
Pavan Jangam	Platform Team Member

Endorsers and Approvers

Name	Role	Purpose	Version	Date	Status

Key Revision History

Version	Date	Description	Author(s)	Status
0.1	24/02/2026	First Draft	Naresh Kala	Draft

Summary

The architecture ensures that classification metadata extracted from Alation flows through a governed dbt pipeline into the Snowflake Governance Database, where it drives automated tag application and masking policy enforcement across all data product tables.

Overview

This document provides a high-level design for automating the ingestion of classification metadata from Alation and applying tagging and masking policies across Snowflake data product tables.

The solution removes manual classification steps and enforces consistent data protection through a governed dbt pipeline.

Objectives

- Automatically ingest classification metadata from Alation via Snowflake Data Share
- Apply Snowflake tags and masking policies without manual intervention
- Centralize all classification records in a single Governance Database
- Orchestrate and schedule the end-to-end pipeline through Airflow
- Maintain full auditability of tagging and masking actions

Current State vs Target State

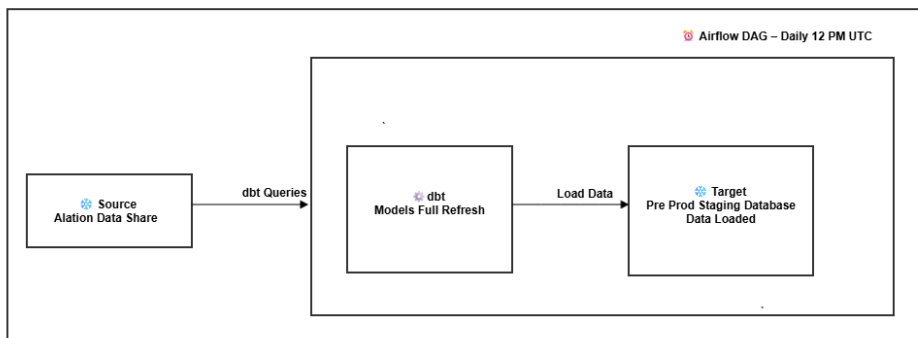
Aspect	Current State (As-Is)	Target State (To-Be)
Classification Entry	Manual entries into CLASSIFICATION_DATA	Automated ingestion from Alation Data Share
Tag Application	dbt macro triggered manually	dbt macros executed automatically on schedule
Masking	Applied manually via tags	Auto-applied by macros based on classification
Processing Scope	Full refresh each run	Incremental — only new or changed records processed

Scheduling	Ad hoc	Daily automated run via Airflow
Audit Trail	Limited	Full logging of all tagging and masking actions

High-Level Architecture

Classification metadata originates in Alation and flows through a Snowflake Database .

Pipeline 1 | Snowflake(Alation data share) Snowflake PreProd Staging Database



1. Source — Snowflake (Alation Data Share)

The source system is Snowflake, with data exposed via Alation Data Share. Alation acts as the data catalog and governance layer on top of Snowflake, providing controlled access to 6 source tables. dbt connects directly to this Snowflake source to query the data.

2.dbt — Models

dbt queries all 6 tables from the Snowflake Alation Data Share. It runs transformation models in full refresh mode — on every execution, the staging tables are fully dropped and rebuilt from source. No incremental logic is applied. The transformed output is written directly into the Snowflake Staging database.

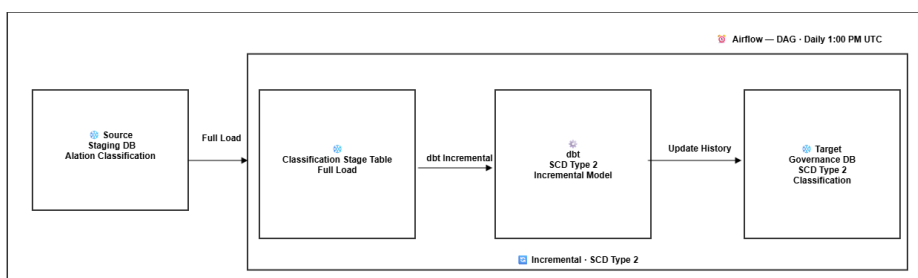
3.Target — Snowflake Staging DB

The target is a Snowflake Staging database. After dbt completes its run, all 6 tables are fully refreshed and available in the staging layer for downstream consumption, reporting, or further transformation.

Astro Airflow — Orchestration

Airflow orchestrates the entire pipeline via a DAG triggered daily at 12:00 PM UTC. Airflow manages task execution order, monitors success or failure, and triggers the dbt job. If any task fails, Airflow halts downstream steps and raises an alert. Both dbt and the Snowflake connection are managed within the Airflow DAG.

Pipeline 2 | Snowflake PreProd Staging Governance DB



1.Source — Snowflake Staging DB

The source for this pipeline is the Snowflake Staging DB populated by Pipeline 1. It contains the Alation classification data that needs to be tracked and governed. This acts as the input for the daily classification load.

2.Stage Table — Full Load

The first step loads the Filtered Alation classification data into a staging table in Snowflake on every run. This is a full load — the stage table is fully truncated and reloaded each time.

3.dbt — SCD Type 2 Incremental Model

dbt reads from the stage table and applies an incremental SCD Type 2 model. It compares incoming records against the Governance DB, identifies new or changed records, and inserts them with effective date tracking — preserving full history. Unchanged records are not re-processed.

4.Target — Governance DB (SCD Type 2)

The Governance DB stores the Alation classification data with full history using SCD Type 2. Each record tracks when it became active and when it was superseded, enabling historical reporting and audit of classification changes over time.

SCD Type 2 — How it works in this pipeline

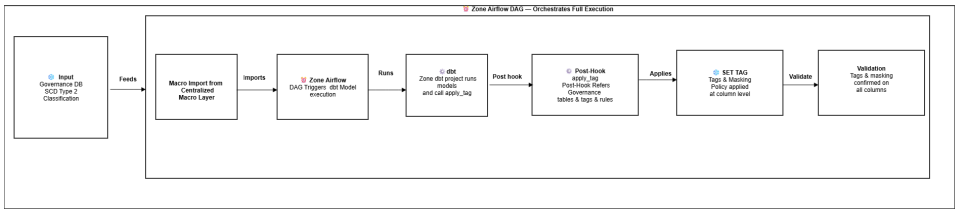
dbt incremental model compares the full stage table snapshot against the Governance DB on each run:

Scenario	Action in Governance DB
New classification record	INSERT new row — effective_start_date = today, is_active = TRUE
Existing record changed	UPDATE old row (effective_end_date = today, is_active = FALSE) + INSERT new row
Record unchanged	No action — row skipped

Astro Airflow — Orchestration

Airflow DAG triggers daily at 1:00 PM UTC. It runs after Pipeline 1 (12:00 PM UTC) ensuring the Snowflake Staging DB is already refreshed before Pipeline 2 begins.

Pipeline 3 | Apply Tag Macro Flow Zone Airflow DAG to Column Classification & Validation



1. Governance DB (Pipeline 2 SCD Type 2) — Input

The Governance DB, populated by Pipeline 2 using SCD Type 2 incremental load, provides the classification rules. It is the reference source that the post-hook queries to determine which tags and masking policies to apply to each column.

2. Macro Imported via packages.yml · dbt deps

The zone dbt project imports the apply_tag macro from the Centralized Macro Layer via packages.yml. Running dbt deps pulls the versioned macro, making it available as a post-hook across all models in the zone project.

3. Zone Airflow DAG — Triggers the Model Job

The Zone Airflow DAG initiates the full execution. It triggers the dbt model job in the zone dbt project, orchestrates task order, and manages success or failure alerting throughout the pipeline.

4. dbt Model Job

The zone dbt project runs its models. Each model materializes in Snowflake under the target schema following Domain__Layer__env naming convention (e.g. Cust__Foundation__dev

5. Post-Hook Runs — Refers Governance DB Tables & Tags

After each model materializes, the apply_tag post-hook fires automatically. It queries the Governance DB to look up the classification rules, tags, and masking policies defined for that model's tables and columns.

6. SET TAG — Column Level Classification Applied

The macro issues ALTER TABLE ... SET TAG and SET MASKING POLICY DDL statements in Snowflake. Tags and masking policies are applied at the column level — transparent to model SQL, enforced by Snowflake at query time based on CURRENT_ROLE().

7. Validation — End of Pipeline

- 1. Verify all expected columns have the correct Snowflake TAG applied
- 2. Confirm role-based access — privileged roles see raw, standard roles see masked
- 3. Airflow marks the DAG run as SUCCESS once validation passes

Key Technical components

Database and Table Details

Strategic masking and classification governance database details.

DB NAME: ALATION__ANALYTICS__SHARE SCHEMA NAME: PUBLIC	TABLE NAMES: ALATION_SET_MEMBER CATALOG_SET_MEMBERSHIP RDBMS_COLUMNS RDBMS_TABLES RDBMS_SCHEMAS RDBMS_DATASOURCES
DB NAME: GOVERNANCE__STG_ALATION__PREPROD SCHEMA NAMES: ALATION	TABLE NAMES: ALATION_SET_MEMBER CATALOG_SET_MEMBERSHIP RDBMS_COLUMNS RDBMS_TABLES RDBMS_SCHEMAS RDBMS_DATASOURCES

Subject Area Developer (HC Developer)	DEV	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access
TESTER / SUPPORT ROLES								
Subject Area Tester (HC Tier)	DEV, SYST, PPTE	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access
Subject Area Tester (CONFIDENTIAL Tier)	DEV, SYST, PPTE	Masked	Plaintext	Plaintext	Plaintext	Masked	Plaintext	No access
Subject Area Tester (PUBLIC / Private Tier)	DEV, SYST, PPTE	Masked	Masked	Plaintext	Plaintext	Masked	Plaintext	No access
SUPPORT ROLES								
Subject Area Support (HC Tier)	DEV, SYST, PPTE, PROD	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access
Subject Area Support (CONFIDENTIAL Tier)	DEV, SYST, PPTE, PROD	Masked	Plaintext	Plaintext	Plaintext	Masked	Plaintext	No access
Subject Area Support (PUBLIC / Private Tier)	DEV, SYST, PPTE, PROD	MASKED	MASKED	Plaintext	Plaintext	MASKED	Plaintext	No access
ANALYST ROLES								
Subject Area Analyst (HC Tier)	PROD	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access	No access
Subject Area Analyst (CONFIDENTIAL Tier)	PROD	MASKED	Plaintext	Plaintext	Plaintext	MASKED	Plaintext	No access
Subject Area Analyst (PUBLIC / Private Tier)	PROD	MASKED	MASKED	Plaintext	Plaintext	MASKED	Plaintext	No access
SERVICE ACCOUNT ROLES								
Airflow Service Role	ALL	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access	No access

Environments

Environment Targets

Environment	Purpose
dev	Developer sandbox — isolated schema per developer
syst	System integration testing
ppte	Pre-production — mirrors production data and roles
prod	Production — full pipeline with live classification enforcement

Testing

Test Scope

Test Type	Scope
Tag Validation	Confirm Snowflake tags applied correctly to all in-scope tables and columns
Masking Validation	Confirm masking policies applied; test role-based masking behavior
Hashing Validation	Verify SHA-256 hashing applied correctly on PK and BK columns
End-to-End Test	Full pipeline:

Transform Database — Personal Schema Data Protection

A single Tag is created and applied once at the Transform database level. Snowflake's tag inheritance automatically propagates the tag down to all schemas, all tables, and all columns inside the database — including any new schemas or tables added in the future. Dynamic Data Masking Policies are created for each data type (VARCHAR, NUMBER, TIMESTAMP) and attached to the Tag. The masking logic uses CURRENT_USER() and CURRENT_SCHEMA() to determine whether the logged-in developer is querying their own schema or another developer's schema.

Final Result

When a developer queries another developer's schema the row count remains visible — only the values are masked as NULL. This is a known Snowflake platform behaviour when using Tag Based Dynamic Data Masking. The actual data content is fully protected and unreadable.

Conclusion

This solution provides a scalable, automated, and governance-driven framework leveraging Alation Data Share and Snowflake capabilities to enforce consistent data protection policies across the organization.